Artificial and Computational Intelligence

Assignment 1 Submission

Problem Statement 0-B - Travelling Agent

Student Name	Gadamsetty V Sai Manoj Kumar
BITS ID	2025MT12100
Selected Problem	Problem Statement 0-B
Algorithms	IDA* and Hill Climbing
Submission Date	11 April 2026

Assigned Problem	Problem Statement 0-B
Graph Goal	Find the shortest route from A to X
Heuristic	Average cost of all simple paths from node to X
Implementation	Executable Python solution with IDA* and Hill Climbing

Verified Program Output

```
Assignment 1 - Problem Statement 0-B
Graph search from Chennai (A) to Hyderabad (X)

Heuristic values (average cost of all simple paths to X):
  h(A) = 22
  h(B) = 27
  h(C) = 31.3333
  h(D) = 19
  h(E) = 24.5
  h(F) = 21.6667
  h(G) = 20.3333
  h(H) = 31.6667
  h(X) = 0

IDA* result:
  Path: A -> D -> G -> X
  Total cost: 19
  Threshold progression: 22, 27.3333
  Nodes expanded: 5
  Max depth reached: 3
  Time complexity: O(b^d) in the worst case
  Space complexity: O(bd) in the worst case

Hill climbing result:
  Path: A -> D -> G -> X
  Total cost: 19
  Nodes expanded: 3
  Max depth reached: 3
  Reached goal: Yes
  Local optimum encountered: Yes
  Time complexity: O(b) per move
  Space complexity: O(b) for neighbor inspection
```

Source Code

```
&amp;quot;&amp;quot;&amp;quot;Assignment 1, Problem Statement 0-B

Travelling Agent on the Chennai-to-Hyderabad graph.

This script implements:
1. IDA* search
2. Hill climbing

The heuristic follows the assignment statement:
the heuristic value h(n) for a node is the average total travel time across
all simple paths from that node to the goal node X.
&amp;quot;&amp;quot;&amp;quot;

from __future__ import annotations

from dataclasses import dataclass
from math import inf
from statistics import fmean
from typing import Dict, Iterable, List, Optional, Set, Tuple


Graph = Dict[str, Dict[str, int]]


GRAPH: Graph = {
    &amp;quot;A&amp;quot;: {&amp;quot;B&amp;quot;: 5, &amp;quot;D&amp;quot;: 3},
    &amp;quot;B&amp;quot;: {&amp;quot;A&amp;quot;: 5},
    &amp;quot;C&amp;quot;: {&amp;quot;G&amp;quot;: 11},
    &amp;quot;D&amp;quot;: {&amp;quot;A&amp;quot;: 3, &amp;quot;E&amp;quot;: 12, &amp;quot;F&amp;quot;: 6, &amp;quot;G&amp;quot;: 4},
    &amp;quot;E&amp;quot;: {&amp;quot;D&amp;quot;: 12, &amp;quot;F&amp;quot;: 7},
    &amp;quot;F&amp;quot;: {&amp;quot;D&amp;quot;: 6, &amp;quot;E&amp;quot;: 7, &amp;quot;H&amp;quot;: 10, &amp;quot;X&amp;quot;: 8},
    &amp;quot;G&amp;quot;: {&amp;quot;C&amp;quot;: 11, &amp;quot;D&amp;quot;: 4, &amp;quot;X&amp;quot;: 12},
    &amp;quot;H&amp;quot;: {&amp;quot;F&amp;quot;: 10},
    &amp;quot;X&amp;quot;: {&amp;quot;F&amp;quot;: 8, &amp;quot;G&amp;quot;: 12},
}

START_NODE = &amp;quot;A&amp;quot;
GOAL_NODE = &amp;quot;X&amp;quot;


@dataclass
class SearchResult:
    path: List[str]
    cost: float
    nodes_expanded: int
    max_depth: int
    thresholds: Optional[List[float]] = None
    local_optimum: bool = False


def _all_simple_paths_with_costs(graph: Graph, start: str, goal: str) -&amp;gt; List[Tuple[List[str], int]]:
    &amp;quot;&amp;quot;&amp;quot;Enumerate all simple paths from start to goal and return their costs.&amp;quot;&amp;quot;&amp;quot;

    paths: List[Tuple[List[str], int]] = []

    def dfs(node: str, current_path: List[str], current_cost: int, visited: Set[str]) -&amp;gt; None:
        if node == goal:
            paths.append((current_path.copy(), current_cost))
            return

        for neighbor, weight in graph[node].items():
            if neighbor in visited:
                continue
            visited.add(neighbor)
            current_path.append(neighbor)
            dfs(neighbor, current_path, current_cost + weight, visited)
            current_path.pop()
            visited.remove(neighbor)

    dfs(start, [start], 0, {start})
    return paths


def build_average_heuristic(graph: Graph, goal: str) -&amp;gt; Dict[str, float]:
    &amp;quot;&amp;quot;&amp;quot;Compute h(n) as the average cost of all simple paths from n to goal.&amp;quot;&amp;quot;&amp;quot;

    heuristic: Dict[str, float] = {}
    for node in graph:
        paths = _all_simple_paths_with_costs(graph, node, goal)
        if not paths:
            heuristic[node] = inf
            continue
        heuristic[node] = fmean(cost for _, cost in paths)
    return heuristic


def hill_climbing(graph: Graph, start: str, goal: str, heuristic: Dict[str, float]) -&amp;gt; SearchResult:
    &amp;quot;&amp;quot;&amp;quot;Perform steepest-ascent hill climbing using the heuristic table.&amp;quot;&amp;quot;&amp;quot;
    current = start
```

```
    path = [current]
    cost = 0.0
    visited = {current}
    nodes_expanded = 0
    max_depth = 0
    encountered_local_optimum = False

    while current != goal:
        nodes_expanded += 1
        neighbors = []
        for neighbor, edge_cost in graph[current].items():
            if neighbor in visited and neighbor != goal:
                continue
            neighbors.append((heuristic[neighbor], edge_cost, neighbor))

        if not neighbors:
            return SearchResult(path=path, cost=cost, nodes_expanded=nodes_expanded, max_depth=max_depth, local_optimum=e

        neighbors.sort(key=lambda item: (item[0], item[1], item[2]))
        improving_neighbors = [item for item in neighbors if item[0] &lt; heuristic[current]]

        if improving_neighbors:
            best_h, edge_cost, best_neighbor = improving_neighbors[0]
        else:
            encountered_local_optimum = True
            best_h, edge_cost, best_neighbor = neighbors[0]

        if best_neighbor in visited and best_neighbor != goal:
            return SearchResult(path=path, cost=cost, nodes_expanded=nodes_expanded, max_depth=max_depth, local_optimum=e

        current = best_neighbor
        path.append(current)
        visited.add(current)
        cost += edge_cost
        max_depth = max(max_depth, len(path) - 1)

    return SearchResult(path=path, cost=cost, nodes_expanded=nodes_expanded, max_depth=max_depth, local_optimum=encounter


def ida_star(graph: Graph, start: str, goal: str, heuristic: Dict[str, float]) -&gt; SearchResult:
    &quot;&quot;&quot;Perform IDA* search on the weighted graph.&quot;&quot;&quot;

    thresholds: List[float] = []
    nodes_expanded = 0
    max_depth = 0

    def search(node: str, g_cost: float, threshold: float, path: List[str], visited: Set[str], depth: int) -&gt; Tuple[Op
        nonlocal nodes_expanded, max_depth

        max_depth = max(max_depth, depth)
        f_cost = g_cost + heuristic[node]
        if f_cost &gt; threshold:
            return None, f_cost

        if node == goal:
            return path.copy(), g_cost

        nodes_expanded += 1
        minimum_excess = inf

        ordered_neighbors = sorted(
            (
                (g_cost + edge_cost + heuristic[neighbor], edge_cost, neighbor)
                for neighbor, edge_cost in graph[node].items()
                if neighbor not in visited
            ),
            key=lambda item: (item[0], item[1], item[2]),
        )

        for _, edge_cost, neighbor in ordered_neighbors:
            visited.add(neighbor)
            path.append(neighbor)
            found_path, next_bound = search(
                neighbor,
                g_cost + edge_cost,
                threshold,
                path,
                visited,
                depth + 1,
            )
            path.pop()
            visited.remove(neighbor)

            if found_path is not None:
                return found_path, _path_cost(graph, found_path)
            minimum_excess = min(minimum_excess, next_bound)

        return None, minimum_excess

    threshold = heuristic[start]
    while True:
        thresholds.append(threshold)
```

```
        visited = {start}
        found_path, result = search(start, 0.0, threshold, [start], visited, 0)
        if found_path is not None:
            return SearchResult(
                path=found_path,
                cost=result,
                nodes_expanded=nodes_expanded,
                max_depth=max_depth,
                thresholds=thresholds,
            )
        if result == inf:
            return SearchResult(path=[], cost=inf, nodes_expanded=nodes_expanded, max_depth=max_depth, thresholds=thresho
        threshold = result


def _path_cost(graph: Graph, path: List[str]) -&gt; float:
    total = 0.0
    for left, right in zip(path, path[1:]):
        total += graph[left][right]
    return total


def _format_path(path: Iterable[str]) -&gt; str:
    return &quot; -&gt; &quot;.join(path)


def print_heuristic_table(heuristic: Dict[str, float]) -&gt; None:
    print(&quot;Heuristic values (average cost of all simple paths to X):&quot;)
    for node in sorted(heuristic):
        value = heuristic[node]
        if value == inf:
            display = &quot;inf&quot;
        else:
            display = f&quot;{value:.4f}&quot;.rstrip(&quot;0&quot;).rstrip(&quot;.&quot;)
        print(f&quot;  h({node}) = {display}&quot;)
    print()


def main() -&gt; None:
    heuristic = build_average_heuristic(GRAPH, GOAL_NODE)

    print(&quot;Assignment 1 - Problem Statement 0-B&quot;)
    print(&quot;Graph search from Chennai (A) to Hyderabad (X)&quot;)
    print()
    print_heuristic_table(heuristic)

    ida_result = ida_star(GRAPH, START_NODE, GOAL_NODE, heuristic)
    hill_result = hill_climbing(GRAPH, START_NODE, GOAL_NODE, heuristic)

    print(&quot;IDA* result:&quot;)
    print(f&quot;  Path: {_format_path(ida_result.path)}&quot;)
    print(f&quot;  Total cost: {ida_result.cost:.0f}&quot;)
    print(f&quot;  Threshold progression: {&#x27;, &#x27;.join(f&#x27;{value:.4f}&#x27;.rstrip(&#x27;0&#x27;).rstrip(&#x2
    print(f&quot;  Nodes expanded: {ida_result.nodes_expanded}&quot;)
    print(f&quot;  Max depth reached: {ida_result.max_depth}&quot;)
    print(&quot;  Time complexity: O(b^d) in the worst case&quot;)
    print(&quot;  Space complexity: O(bd) in the worst case&quot;)
    print()

    print(&quot;Hill climbing result:&quot;)
    print(f&quot;  Path: {_format_path(hill_result.path)}&quot;)
    print(f&quot;  Total cost: {hill_result.cost:.0f}&quot;)
    print(f&quot;  Nodes expanded: {hill_result.nodes_expanded}&quot;)
    print(f&quot;  Max depth reached: {hill_result.max_depth}&quot;)
    print(f&quot;  Reached goal: {&#x27;Yes&#x27; if hill_result.path and hill_result.path[-1] == GOAL_NODE else &#x27;No
    print(f&quot;  Local optimum encountered: {&#x27;Yes&#x27; if hill_result.local_optimum else &#x27;No&#x27;}&quot;)
    print(&quot;  Time complexity: O(b) per move&quot;)
    print(&quot;  Space complexity: O(b) for neighbor inspection&quot;)


if __name__ == &quot;__main__&quot;:
    main()
```

Result Summary

Algorithm	Observed Result	Time Complexity	Space Complexity
IDA*	A -> D -> G -> X	O(b^d) worst case	O(bd) worst case
Hill Climbing	A -> D -> G -> X	O(b) per move	O(b) for neighbor inspection